

Q1. Design a class hierarchy: Vehicle to Car to ElectricCar.

```
class Vehicle {
    protected String brand;

    Vehicle(String brand) {
        this.brand = brand;
    }

    void displayInfo() {
        System.out.println("Brand: " + brand);
    }
}

class Car extends Vehicle {
    protected int numDoors;

    Car(String brand, int numDoors) {
        super(brand);
        this.numDoors = numDoors;
    }

    @Override
    void displayInfo() {
        super.displayInfo();
        System.out.println("Doors: " + numDoors);
    }
}

class ElectricCar extends Car {
    private int batteryRangeKm;

    ElectricCar(String brand, int numDoors, int batteryRangeKm) {
        super(brand, numDoors);
        this.batteryRangeKm = batteryRangeKm;
    }

    @Override
```

```

    void displayInfo() {
        super.displayInfo();
        System.out.println("Battery Range: " + batteryRangeKm + " km");
    }
}

public class VehicleHierarchyDemo {
    public static void main(String[] args) {
        ElectricCar ec = new ElectricCar("Tesla", 4, 500);
        ec.displayInfo();
    }
}

```

Vehicle defines the most general attributes (brand), Car extends Vehicle by adding car-specific detail (numDoors), and ElectricCar further extends Car with electric-specific detail (batteryRangeKm). Each level reuses and builds upon the fields and behavior of the level above it using inheritance.

Q2. Create a superclass Employee and subclass Manager.

```

class Employee {
    protected String name;
    protected double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Salary: " + salary);
    }
}

class Manager extends Employee {
    private int teamSize;

    Manager(String name, double salary, int teamSize) {
        super(name, salary);
        this.teamSize = teamSize;
    }
    @Override

```

```

    void displayDetails() {
        super.displayDetails();
        System.out.println("Team Size: " + teamSize);
    }
}

public class EmployeeManagerDemo {
    public static void main(String[] args) {
        Manager m = new Manager("Priya", 75000, 8);
        m.displayDetails();
    }
}

```

Manager inherits the name and salary fields and behavior from Employee, and adds its own field, teamSize, specific to managers, demonstrating how a subclass can extend a general superclass with more specialized data.

Q3. Override a method in Java and explain the use of super.

```

class Shape {
    void describe() {
        System.out.println("This is a generic shape.");
    }
}

class Square extends Shape {
    private double side;

    Square(double side) {
        this.side = side;
    }

    @Override
    void describe() {
        super.describe(); // calls the parent class version first
        System.out.println("Specifically, a square with side " + side);
    }
}

public class SuperDemo {
    public static void main(String[] args) {
        Square sq = new Square(5);
        sq.describe();
    }
}

```

```
}  
}
```

The describe method is overridden in Square to provide more specific behavior than the version in Shape. The super keyword is used inside the overriding method to explicitly call the parent class's version of describe before adding additional, subclass-specific output. The super keyword can similarly be used to call a superclass's constructor (as seen in the earlier Manager and ElectricCar examples), or to access a hidden superclass field.